PROGRAMAÇÃO I

ESTRUTURAS DE DADOS EM PYTHON

VOLUME II • CADERNO 03

ESTRUTURAS COMBINADAS E JSON

Como representar produtos, estoque e pedidos em estruturas que podem ser gravadas em arquivo.

COMO ESTUDAR Comece pelo problema concreto, acompanhe o estado dos dados antes e depois de cada operação e só então leia o código. Execute os exemplos na ordem. Quando aparecer um termo técnico, ele será definido no próprio capítulo. Não é necessário decorar comandos antes de entender o comportamento.

CONTEXTO DIDÁTICO Os exemplos acompanham uma rede fictícia de lojas, com produtos, clientes, estoque, pedidos e atendimento. Os registros são exemplos de aula, não dados comerciais reais. Este caderno pressupõe os fundamentos de Python e o Caderno 01 (listas).

Série de cadernos independentes e complementares • Texto explicativo, exemplos executáveis e roteiros físicos de representação.

Mapa de aprendizagem deste caderno

Este caderno integra as coleções estudadas separadamente, apresenta listas de dicionários e matrizes, explica a diferença entre memória e persistência e desenvolve leitura, consulta, alteração e gravação de dados em arquivos JSON.

As representações em tabelas mostram o que o computador armazena; as demonstrações físicas mostram como uma pessoa poderia simular as mesmas operações com cartões ou colegas em posições fixas. **O número da posição não se move: o elemento é que muda.**

1. Um produto, muitos produtos e muitas informações

Uma lista de nomes é útil quando precisamos apenas percorrer nomes. Porém um catálogo precisa guardar, para cada produto, seu código, preço, categoria e estoque. Uma solução intuitiva é usar **um dicionário para cada produto** e **uma lista para reunir todos os produtos**. Essa combinação é uma das formas mais frequentes de representar dados em Python.

```
produtos = [
    {"codigo": 101, "nome": "Shampoo", "preco": 24.90},
    {"codigo": 102, "nome": "Sabonete", "preco": 18.50},
    {"codigo": 103, "nome": "Protetor", "preco": 59.90}
]
print(produtos[1]["nome"])  # Sabonete
```

Leia a expressão por etapas: `produtos[1]` retorna o segundo registro, um dicionário; em seguida, `["nome"]` lê o campo nome desse registro. A linguagem permite encadear acessos porque cada operação retorna um valor que pode ser consultado novamente.

Índice da lista	Código	Nome	Preço
0	101	Shampoo	24.90
1	102	Sabonete	18.50
2	103	Protetor	59.90

SIMULAÇÃO NA SALA Cada aluno representa um produto e fica em uma posição numerada da fila. Em sua mão, segura uma ficha com campos nome, código e preço. O número no chão representa o índice da lista; a ficha representa o dicionário. Mudar o aluno de posição é diferente de mudar o preço escrito na ficha.

2. Percorrer registros e selecionar informações

O percurso mais simples usa `for` para obter um dicionário de cada vez. Na repetição, a variável `produto` passa a referenciar sucessivamente os registros da lista. Uma condição permite selecionar apenas os que atendem a determinado critério.

```
produtos = [
    {"nome": "Shampoo", "estoque": 18},
    {"nome": "Protetor", "estoque": 9},
    {"nome": "Perfume", "estoque": 5}
]
for produto in produtos:
    if produto["estoque"] < 10:
        print(produto["nome"])
```

Saída: Protetor e Perfume, cada um em uma linha. O comando não apaga os demais produtos; ele apenas decide quais serão mostrados. Selecionar, transformar e remover são operações distintas que não devem ser confundidas.

Construir uma nova lista com os selecionados

```
baixos = []
for produto in produtos:
    if produto["estoque"] < 10:
        baixos.append(produto)
print(len(baixos))  # 2
```

Aqui a nova lista guarda referências aos mesmos dicionários. Se alterarmos `baixos[0]["estoque"]`, alteraremos também o dicionário correspondente visto pela lista `produtos`. Isso acontece porque os registros internos não foram copiados.

3. Referências, cópias rasas e cópias profundas

Uma dificuldade comum é imaginar que atribuir uma variável cria automaticamente uma cópia independente. Em Python, a atribuição normalmente cria **outro nome para o mesmo objeto**. Por isso, a alteração em uma estrutura mutável pode aparecer em lugares diferentes do programa.

```
original = [{"nome": "Shampoo", "estoque": 18}]
outra = original
outra[0]["estoque"] = 10
print(original[0]["estoque"])  # 10
```

Usar `original.copy()` cria uma lista externa nova, mas seus dicionários internos continuam compartilhados. Esse é o significado de cópia **rasa**. Para obter uma cópia independente também dos dicionários, `copy.deepcopy()` é uma alternativa.

```
from copy import deepcopy
original = [{"nome": "Shampoo", "estoque": 18}]
copia = deepcopy(original)
copia[0]["estoque"] = 10
print(original[0]["estoque"])  # 18
```

Expressão	Nova lista externa?	Novos dicts internos?
outra = original	Não	Não
outra = original.copy()	Sim	Não
outra = deepcopy(original)	Sim	Sim

EXEMPLO FÍSICO Uma ficha de produto pode ser compartilhada por duas equipes. Se ambas apontam para a mesma ficha, alterar o preço escrito muda o que todas enxergam. Fazer uma fotocópia independente da ficha corresponde melhor ao efeito de uma cópia profunda.

4. Listas aninhadas e matrizes de estoque

Podemos usar uma **lista de listas** quando os dados são naturalmente organizados em linhas e colunas. Imagine duas lojas, cada uma com o estoque de três produtos, sempre na mesma ordem. A linha identifica a unidade e a coluna identifica o produto.

Loja / Produto	Shampoo	Sabonete	Protetor
Centro	18	27	9
Eldorado	10	14	6

```
estoques = [
    [18, 27, 9],   # Centro
    [10, 14, 6]    # Eldorado
]
```

```
print(estoques[0][2])  # 9: Centro, Protetor
print(estoques[1][0])  # 10: Eldorado, Shampoo
```

A expressão `estoques[0][2]` deve ser lida assim: primeiro selecione a linha 0; depois, dentro dela, selecione a coluna 2. Os dois índices são independentes e começam em zero.

```
for indice_loja, linha in enumerate(estoques):
    total = sum(linha)
    print(indice_loja, total)
# 0 54
# 1 30
```

Esse exemplo pressupõe que cada linha representa a mesma sequência de produtos. Se a ordem dos produtos mudar em apenas uma unidade, uma matriz posicional pode gerar erros de interpretação. Para cadastros heterogêneos, a estrutura por dicionários costuma ser mais explícita.

5. O que é JSON e o que ele não é

JSON significa **JavaScript Object Notation** e é um formato textual para representar dados estruturados. Um arquivo `.json` contém texto organizado segundo regras específicas. Ele não é uma estrutura própria de Python; é um formato de intercâmbio que Python pode interpretar e gerar.

Exemplo de texto JSON (não é código Python).

```
{
  "nome": "Shampoo",
  "preco": 24.90,
  "estoque": 18,
  "ativo": true,
  "desconto": null
}
```

No JSON	Depois de carregar no Python
Objeto { ... }	dict
Array [...]	list
Texto	str
Número inteiro / decimal	int / float
true / false	True / False
null	None

No JSON, as chaves de objeto devem ser textos entre aspas duplas. Também é importante escrever `true`, `false` e `null` em minúsculas. No Python, os equivalentes são `True`, `False` e `None`. A distinção evita erros ao copiar dados de um arquivo para o código.

NÃO CONFUNDIR **JSON** é um documento textual persistente; **dict** e **list** são estruturas que o programa usa em memória. O arquivo pode ser carregado como estruturas Python, e essas estruturas podem ser gravadas novamente como JSON.

6. Carregar um arquivo JSON com segurança

O módulo padrão `json` oferece a função `json.load()`, que interpreta o conteúdo de um arquivo aberto e devolve a estrutura Python correspondente. O exemplo usa o arquivo `lojas_rede_exemplo.json`, fornecido junto deste caderno.

```
import json
```

```
with open("lojas_rede_exemplo.json", "r", encoding="utf-8") as arquivo:
    dados = json.load(arquivo)

print(type(dados).__name__)  # dict
print(len(dados["produtos"]))  # 8
print(dados["produtos"][0]["nome"])  # Shampoo Nutritivo
```

O bloco `with` fecha o arquivo automaticamente ao terminar. O argumento `encoding="utf-8"` evita problemas com caracteres acentuados na leitura. A variável `dados` passa a referenciar a estrutura carregada, e seu tipo depende da raiz do documento JSON.

O que fazer quando o arquivo não existe ou está inválido

```
import json
try:
    with open("lojas_rede_exemplo.json", encoding="utf-8") as f:
        dados = json.load(f)
except FileNotFoundError:
    print("Arquivo não encontrado.")
except json.JSONDecodeError:
    print("O arquivo não contém JSON válido.")
```

O tratamento de exceções não dispensa a atenção ao conteúdo: um documento pode ter JSON válido, mas não possuir as chaves esperadas pelo programa. Nesses casos, devemos validar a estrutura e as informações antes de utilizá-las.

7. Consultar dados reais de uma coleção estruturada

Com o catálogo carregado, podemos combinar laços, condições, funções e dicionários para produzir relatórios. Este exemplo localiza produtos com estoque inferior a um limite e calcula o valor financeiro dos itens selecionados.

```
import json
with open("lojas_rede_exemplo.json", encoding="utf-8") as f:
    dados = json.load(f)

limite = 12
for p in dados["produtos"]:
    if p["estoque"] < limite:
        valor = p["preco"] * p["estoque"]
        print(f"{p['nome']}: {p['estoque']} un. / R$ {valor:.2f}")
```

O código percorre produtos, não lojas nem pedidos. A cada repetição, `p` é um dicionário de produto. O teste usa a quantidade em estoque; o cálculo multiplica essa quantidade pelo preço; a saída apresenta apenas os produtos que passaram na condição. O filtro é uma decisão, não uma alteração do arquivo.

Localizar um registro pelo código

```
def buscar_produto(produtos, codigo):
    for produto in produtos:
        if produto["codigo"] == codigo:
            return produto
    return None

encontrado = buscar_produto(dados["produtos"], 103)
print(encontrado["nome"] if encontrado else "Não encontrado")
```

A função retorna o primeiro produto com código igual ao procurado. Se não existir, devolve `None`. Mais adiante veremos que essa estratégia é chamada de **busca linear**, pois examina os elementos um por um.

8. Alterar em memória e gravar uma cópia do JSON

Quando o programa carrega um arquivo, passa a trabalhar com valores na memória. Alterar esses valores não modifica o conteúdo do arquivo de origem. Para salvar, precisamos executar uma operação de escrita explícita.

```
import json
from copy import deepcopy

with open("lojas_rede_exemplo.json", encoding="utf-8") as f:
    dados = json.load(f)

copia = deepcopy(dados)
copia["produtos"][0]["estoque"] += 5

with open("lojas_rede_alterado.json", "w", encoding="utf-8") as f:
    json.dump(copia, f, ensure_ascii=False, indent=2)
```

Neste exemplo, o arquivo original continua igual. O programa cria uma cópia dos dados, acrescenta cinco unidades ao primeiro produto e grava o resultado em outro arquivo. `ensure_ascii=False` preserva letras acentuadas, e `indent=2` torna o texto mais legível.

Operação	Onde acontece?
json.load()	Lê o texto do arquivo e constrói objetos em memória.
dados["produtos"][0]["estoque"] += 5	Altera um valor em memória.
json.dump(..., arquivo_saida)	Grava o estado atual em um arquivo.
Reabrir o arquivo original	Lê novamente os valores originais não gravados nele.

BOA PRÁTICA Ao estudar, grave alterações em uma cópia, não no conjunto de dados original. Em sistemas reais, gravar um arquivo diretamente exige cuidados adicionais contra interrupção e gravação parcial, além de validar os dados antes de substituí-lo.

9. Um pequeno relatório organizado por função

Ao reunir as operações em funções, o programa se torna mais fácil de compreender e reutilizar. A função a seguir recebe a lista, calcula um resultado e não precisa conhecer o arquivo de onde os dados vieram. Essa separação diminui o acoplamento entre leitura e processamento.

```
def valor_total_estoque(produtos):
    total = 0
    for p in produtos:
        total += p["preco"] * p["estoque"]
    return total

def categorias_distintas(produtos):
    return {p["categoria"] for p in produtos}

print(f"R$ {valor_total_estoque(dados['produtos']):.2f}")
print(sorted(categorias_distintas(dados["produtos"])))
```

Observe que `categorias_distintas()` utiliza compreensão de conjuntos: para cada produto, recolhe a categoria; como o resultado é `set`, categorias repetidas aparecem apenas uma vez. A função

`sorted()` cria uma lista ordenada com os nomes para exibição, sem criar índices no conjunto original.

10. Quando utilizar JSON e quando outra solução é melhor

JSON é conveniente para configuração, intercâmbio e arquivos pequenos ou moderados que podem ser carregados inteiros. Não é substituto direto de um banco de dados quando existem muitos usuários gravando ao mesmo tempo, grandes volumes de registros ou consultas complexas.

Necessidade	Possível solução
Compartilhar um catálogo pequeno entre programas	JSON
Manipular uma lista temporária durante o programa	list/dict em memória
Consultar milhares de pedidos com várias pessoas alterando	Banco de dados
Guardar configurações simples	JSON
Trocar dados com uma API	JSON em uma requisição/resposta

Nos próximos cadernos, esses mesmos dados serão utilizados para procurar produtos, ordenar preços e estudar estruturas que controlam a sequência de atendimento e processamento. O valor de aprender as coleções está justamente em reconhecer que elas servem a tarefas reais.

Para guardar deste caderno

- **Lista de dicionários**: a lista reúne registros, e o dicionário identifica campos por chave.
- **Lista de listas**: útil para dados regulares organizados em linhas e colunas, desde que os índices tenham significado estável.
- **JSON**: texto que pode ser interpretado como estruturas Python; carregar não é o mesmo que salvar.
- Cópias rasas e profundas produzem comportamentos diferentes quando há estruturas mutáveis aninhadas.

Referências de consulta

Python Software Foundation. The Python Tutorial: Data Structures. https://docs.python.org/3/tutorial/datastructures.html

Python Software Foundation. Biblioteca padrão: collections e json. https://docs.python.org/3/library/collections.html ; https://docs.python.org/3/library/json.html

VisuAlgo. Visualizações e explicações de algoritmos. https://visualgo.net/en

Harvard University. CS50x 2026, Week 3 — Algorithms. https://cs50.harvard.edu/x/weeks/3/

Bhargava, Aditya. Entendendo Algoritmos (Grokking Algorithms), 2ª edição. Manning Publications. Referência de abordagem visual; exemplos e explicações deste caderno são originais.